Week 4 • Class 1

Agentic Systems

LLM Planner + Tools + State + Control Loop

From "AI that knows" → "AI that decides and acts"

Today we move beyond retrieval-only systems. **Agents** use an LLM to plan the next step, call tools, update state, and repeat until the task is done.

You will build a live Support Resolution Agent that retrieves policy, decides next actions, and drafts a final response.

Class shift

- Week 1 → Model
- Week 2 → Control
- Week 3 → Knowledge
- **Week 4 → Agency**

Why agents exist

When prompts and RAG are not enough

Prompt System

- Generate text
- Single response
- No external action

Summarize / rewrite / translate

RAG System

- Retrieve knowledge
- Ground answer in context
- Still answer-oriented

PDFs / docs / internal knowledge

Agentic System

- Choose next action
- Use tools
- Update state and continue

Multi-step decision processes

A workflow follows a map. An agent decides where to go next.

What an agent really is

An agent is a controlled decision loop over tools and state.

User Goal → Planner LLM → Choose Tool → Run Tool → Update State → Next Step? → Final Answer

Goal
What is the user trying to achieve?

Tools
What capabilities are available?

State
What has already happened?

Control
What constraints keep it safe?

Running use case for the complete class

Support Resolution Agent

User request:
“A customer wants a refund after 2 weeks and says the product was damaged. What should we do?”

1. **Classify issue**
2. Retrieve refund policy
3. Retrieve damage policy
4. Recommend action
5. **Draft final response**

This is a perfect agent problem because one answer is not enough — the system must decide and sequence multiple actions.

Tool layer: capabilities in Python

Tools are usually just Python functions. The agent chooses when to use them.

```
def get_refund_policy():
    return "Refunds are allowed within 30 days with receipt."

def get_damage_policy():
    return "Damaged items can be replaced or issued store credit."

def classify_issue(user_input):
    text = user_input.lower()

    if "refund" in text and "damage" in text:
        return "refund_damage_case"
    elif "refund" in text:
        return "refund_case"
    elif "damage" in text:
        return "damage_case"
    else:
        return "general_case"

def recommend_action(issue_type):
    if issue_type == "refund_damage_case":
        return "Offer replacement or refund depending on receipt and item status."
    elif issue_type == "refund_case":
        return "Check refund window and receipt."
    elif issue_type == "damage_case":
        return "Offer replacement or store credit."
    else:
        return "Escalate for manual review."
```

These are capabilities — not an agent yet.

Planner design: JSON becomes operational

The LLM is not answering the user directly. It is choosing the next action in machine-readable form.

Planner prompt asks for:

- thought
- next_tool
- tool_input
- done

Why this matters

Week 2 JSON now controls agent behavior, not just output formatting.

```
{
  "thought": "Need issue type before policy lookup",
  "next_tool": "classify_issue",
  "tool_input": "customer wants refund after 2 weeks and says item was damaged",
  "done": false
}
```

Planner prompt: LLM chooses the next step

The LLM is no longer answering the user directly. It returns the next action in JSON.

```
from openai import OpenAI
import json

client = OpenAI()

def plan_next_step(user_request, state):
    prompt = f"""
You are an AI support agent planner.

Available tools:
- classify_issue
- get_refund_policy
- get_damage_policy
- recommend_action
- finalize_answer

User request:
{user_request}

Current state:
{json.dumps(state, indent=2)}

Return ONLY valid JSON with:
{
  "thought": "short reasoning",
  "next_tool": "tool name",
  "tool_input": "input for tool",
  "done": true or false
}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0
    )

    return json.loads(response.choices[0].message.content)
```

Week 2 JSON becomes operational in Week 4.

Tool runner: planner decides, Python executes

Map the planner's decision to real functions.

```
def run_tool(tool_name, tool_input):
    if tool_name == "classify_issue":
        return classify_issue(tool_input)

    elif tool_name == "get_refund_policy":
        return get_refund_policy()

    elif tool_name == "get_damage_policy":
        return get_damage_policy()

    elif tool_name == "recommend_action":
        return recommend_action(tool_input)

    else:
        return "Unknown tool"
```

The planner decides. Python executes.

State: the hidden premium concept

Without state the agent forgets what it has already done. With state the planner can build on prior steps.

```
state = {
  "issue_type": None,
  "refund_policy": None,
  "damage_policy": None,
  "recommended_action": None,
  "steps": []
}
```

Think of state as the agent's working memory.

Planner reads state

Tools update state

Next decision depends on state

State is how the agent remembers progress.

Agent loop: where it becomes real

This is the control loop that turns tools into an agentic system.

The loop continues until the task is complete or the step limit is reached.

The actual agent loop

This is where tools, state, and planning become a real agent.

```
def support_resolution_agent(user_request, max_steps=5):
    state = {
        "issue_type": None,
        "refund_policy": None,
        "damage_policy": None,
        "recommended_action": None,
        "steps": []
    }

    for step in range(max_steps):
        plan = plan_next_step(user_request, state)
        next_tool = plan["next_tool"]
        tool_input = plan["tool_input"]
        done = plan["done"]

        state["steps"].append({
            "step": step + 1,
            "thought": plan["thought"],
            "tool": next_tool,
            "tool_input": tool_input
        })

        if done or next_tool == "finalize_answer":
            break

        result = run_tool(next_tool, tool_input)

        if next_tool == "classify_issue":
            state["issue_type"] = result
        elif next_tool == "get_refund_policy":
            state["refund_policy"] = result
        elif next_tool == "get_damage_policy":
            state["damage_policy"] = result
        elif next_tool == "recommend_action":
            state["recommended_action"] = result
```

An agent is a controlled reasoning loop over tools and state.

Final answer synthesis + live run

The agent uses collected state to draft a grounded final response.

```
final_prompt = f"""
You are a customer support AI assistant.

User request:
{user_request}

State collected:
{json.dumps(state, indent=2)}

Write a final helpful response to the user.
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": final_prompt}],
    temperature=0.0
)

return {
    "state": state,
    "final_answer": response.choices[0].message.content
}

result = support_resolution_agent(
    "A customer wants a refund after 2 weeks and says the product was damaged."
)
print(result["final_answer"])
print(json.dumps(result["state"], indent=2))
```

This is not a chatbot anymore. This is a decision system.

The LLM is not the whole agent.
The LLM is the planner inside the agent.

Guardrails for agentic systems

Max step limit — Prevents infinite loops

Allowed tool list — Prevents unsafe or random tool calls

State validation — Prevents incomplete final answers

Human escalation — Fallback when confidence is low

Why premium systems need control

The more AI can do, the more constrained the system must be.

Capability without control is dangerous.

High-value real-world agent use cases

Support resolution agent

Classify issue → retrieve policy → recommend action → draft reply

HR assistant agent

Identify leave type → fetch handbook rule → suggest next steps

CRM follow-up agent

Read lead notes → choose next action → draft personalized follow-up

Incident response agent

Summarize issue → retrieve runbook → recommend remediation

These are not prompt tricks. These are software systems.

Week 4 Class 1 assignment — upgrade the assistant again

Do not build a new app. Upgrade the same assistant from semantic retrieval → planner + tool-using agent.

Level 1 Add 2–3 tools and manual execution

Level 2 Use the LLM to choose the next tool with JSON

Level 3 Run a multi-step loop with state

Level 4 Return steps, tools used, final answer, and final state

Submission: Python file + screenshot of output + 3–5 line reflection on what the planner did.

Close

RAG makes AI informed.
Agents make AI operational.

This is where AI stops being just a smart responder and starts becoming a decision system.